

Arquitectura Técnica y Especificaciones de Diseño para la Adaptación en Audiogame Binaural de Subway Surfers (2012) en C# .NET

1. Investigación Profunda de Mecánicas y Lógica de Juego (2012)

Estructura del Sistema de Carriles y Física de Movimiento

El diseño mecánico de *Subway Surfers* se fundamenta en un espacio tridimensional estructurado a lo largo de tres carriles discretos e independientes¹. En el eje horizontal (X), la posición del jugador está confinada a tres valores absolutos: Carril Izquierdo ($X = -3.0 \text{ m}$), Carril Central ($X = 0.0 \text{ m}$) y Carril Derecho ($X = +3.0 \text{ m}$). Las transiciones entre carriles adyacentes no se efectúan de forma instantánea; el motor procesa una interpolación suave (*Ease-Out*) con una duración nominal de entre 150 ms y 200 ms , lo que exige prever la trayectoria cuando se ejecutan esquivas consecutivas.

El movimiento longitudinal a lo largo del eje Z se rige por un vector de velocidad progresiva¹. El personaje inicia la carrera con una velocidad base $v_0 = 10.0 \text{ m/s}$ (36 km/h). Con el transcurso del tiempo t (medido en segundos), la velocidad se incrementa siguiendo una función de aceleración logarítmica que estabiliza la curva de dificultad y previene saltos de velocidad inmanejables, alcanzando un techo cinemático de $v_{\max} = 30.0 \text{ m/s}$ (108 km/h) aproximadamente a los 300 segundos de recorrido ininterrumpido³.

$$v(t) = \min(v_{\max}, v_0 + \alpha \cdot \ln(1 + \beta \cdot t)) \quad (1)$$

En esta formulación, $\alpha \approx 3.5$ y $\beta \approx 0.05$ representan las constantes de escalado ajustadas para la progresión del juego original. En el eje vertical (Y), la física de salto genera

una parábola gravitacional ($g = -19.6 \text{ m/s}^2$) que eleva la caja de colisión del avatar hasta una altura máxima de **2.5 m** con un tiempo total de permanencia en el aire de **600 ms**. Por su parte, la acción de agacharse o rodar reduce la altura de la caja de colisión a **0.8 m** respecto al suelo durante una ventana fija de **500 ms**.

Parámetro Dinámico	Valor Inicial (t=0s)	Valor Medio (t=120s)	Límite Máximo (t≥300s)
Velocidad Longitudinal (v)	10.0 m/s (36.0 km/h)	21.5 m/s (77.4 km/h)	30.0 m/s (108.0 km/h)
Aceleración Instantánea ($\frac{dv}{dt}$)	0.08 m/s ²	0.03 m/s ²	0.00 m/s ²
Duración Transición de Carril	200 ms	170 ms	150 ms
Tiempo de Aire en Salto Base	600 ms	600 ms	600 ms
Duración de Agachado / Rodar	500 ms	500 ms	500 ms

Fichas Técnicas de Obstáculos

Los obstáculos constituyen las entidades físicas primarias con las que interactúa el sistema de detección de colisiones del motor. Se diferencian cuatro clases estructurales:

1. **Vallas Bajas (Hurdles / Low Barriers):** Presentan una altura física de **1.0 m** desde la superficie de la vía. Requieren obligatoriamente la ejecución de un salto. Intentar atravesarlas en posición erguida o rodando activa una colisión de impacto frontal o tropezón.
2. **Vallas Altas (High Barriers / Overhead Obstacles):** Bloquean el espacio aéreo desde una altura de **1.2 m** hasta los **2.8 m**, dejando una luz inferior de **1.2 m** libre respecto al suelo. Exigen que el personaje se agache o ruede. Saltar contra una valla alta

provoca un impacto directo contra el travesaño superior.

3. **Trenes Estáticos:** Vagonetas de ferrocarril inmóviles con dimensiones estándar de **18.0 m** de longitud, **2.6 m** de ancho y **3.8 m** de altura. Se pueden abordar

frontalmente únicamente si disponen de una rampa de acceso inclinado (15°), o mediante un salto desde otro elemento elevado o usando potenciadores de gran impulso.

4. **Trenes en Movimiento:** Vagonetas dinámicas que se desplazan en sentido contrario a la dirección del jugador a una velocidad propia constante de $v_{\text{tren}} = 12.0 \text{ m/s}$ (43.2 km/h). La velocidad relativa de aproximación resulta de la suma vectorial de ambas velocidades ($v_{\text{relativa}} = v + v_{\text{tren}}$), alcanzando los 42.0 m/s en la fase de máxima velocidad del juego.

Tiempos de Reacción y Ventanas Temporales

La viabilidad técnica de un audiogame accesible depende del tiempo del que dispone el jugador para procesar la señal auditiva espacializada y ejecutar la respuesta en el teclado.

Considerando una tasa de refresco estándar de 60 fotogramas por segundo (16.67 ms por fotograma), se establecen las siguientes ventanas de reacción según la velocidad del avatar:

Obstáculo	Distancia de Generación	Velocidad Jugador	Velocidad Relativa	Tiempo de Reacción	Marcos de Tiempo (Frames a 60 FPS)
Valla Baja / Alta	20.0 m	10.0 m/s	10.0 m/s	2000 ms	120 frames
Valla Baja / Alta	20.0 m	30.0 m/s	30.0 m/s	667 ms	40 frames
Tren Estático	30.0 m	20.0 m/s	20.0 m/s	1500 ms	90 frames
Tren en Movimiento	40.0 m	10.0 m/s	22.0 m/s	1818 ms	109 frames
Tren en Movimiento	40.0 m	30.0 m/s	42.0 m/s	952 ms	57 frames

o					
---	--	--	--	--	--

Comportamiento de Potenciadores y Patrones de Monedas

Los potenciadores (*Power-ups*) modifican las leyes dinámicas del juego o la interacción con las monedas dispersas en la vía:

- **Imán (Magnet):** Genera un campo de atracción electromagnético con un radio efectivo de 12.0 m ⁴. Recolecta automáticamente todas las monedas presentes en los tres carriles sin requerir un desplazamiento lateral. Posee una duración activa de **10 segundos**.
- **Jetpack:** Propulsa al jugador verticalmente hasta una altitud de seguridad de $Y = 10.0\text{ m}$ durante **12 segundos**. En este plano aéreo no existen obstáculos. Las monedas se disponen en formaciones continuas de tres carriles, arcos parabólicos y patrones ondulatorios que sirven como guía auditiva de avance.
- **Zapatillas (Super Sneakers):** Modifican el impulso vertical del salto, incrementando la altura máxima a 4.5 m y extendiendo el tiempo de vuelo a **900 ms**. Esto permite saltar directamente sobre el frontal de trenes estáticos o de vallas altas.
- **Hoverboard:** Escudo de protección con una duración de **30 segundos**⁴. En caso de colisión fatal contra un obstáculo, el Hoverboard se destruye otorgando una ventana de invulnerabilidad de **1.5 segundos** e interrumpiendo la secuencia de captura⁴.
- **Multiplicador 2x:** Duplica el factor de conversión de distancia recorrida a puntos de juego sin alterar las variables físicas de velocidad o aceleración⁴.

Las monedas se organizan en secuencias estandarizadas: hileras rectas de 5 a 15 unidades al nivel de las vías, arcos parabólicos que siguen la trayectoria ideal de salto sobre vallas bajas, y líneas elevadas a lo largo del techo de los trenes.

Sistema de Persecución: Inspector y Perro

El inspector y su perro constituyen el mecanismo de penalización y estado crítico³. La distancia nominal de persecución se sitúa fuera del plano directo de peligro ($d > 15.0\text{ m}$ por detrás del jugador). El motor gestiona dos estados de colisión:

1. **Primer Tropezón (Estado de Advertencia):** Se produce al impactar de forma rozante contra el lateral de una valla o el costado de un vagoneta sin frenar en seco el avance. La distancia del inspector se reduce instantáneamente a 1.5 m durante un periodo de **10 segundos**. En este estado, el plano de audio posterior debe reproducir pasos pesados, jadeos caninos y respiración agitada en la zona inmediata detrás del oyente. Si

el jugador no comete nuevos errores en **10 segundos** , la distancia se restablece al valor nominal de seguridad.

2. **Choque Fatal:** Se desencadena al colisionar frontalmente contra un tren o pared, o al sufrir un segundo tropezón rozante mientras el inspector se halla en el rango de **1.5 m**

⁴. Esto finaliza la partida e inicia la animación sonora de captura.

2. Inventario Completo de SFX y Guía de Extracción del APK Original

Inventario Estructurado de Archivos de Audio (SFX)

Para mantener la fidelidad sonora del juego original, se requiere un repertorio organizado de efectos limpios sin música de fondo superpuesta:

Categoría	Archivo de Audio Sugerido	Función Mecánica y Evento Asociado
Locomoción	step_ballast_1.wav .. 4.wav	Pasos sobre la grava o balasto de las vías (carril normal).
Locomoción	step_metal_1.wav .. 4.wav	Pasos resonantes sobre el techo metálico de los trenes.
Locomoción	swipe_lane.wav	Transición lateral rápida entre carriles adyacentes.
Locomoción	jump_launch.wav	Impulso e inicio de elevación en un salto estándar.
Locomoción	jump_sneakers.wav	Impulso neumático al saltar con Super Sneakers.
Locomoción	roll_slide.wav	Deslizamiento del cuerpo sobre el suelo al rodar.
Obstáculos	train_approaching_loop.wav	Bucle dinámico del motor/rodadura de tren en movimiento.

Obstáculos	train_horn.wav	Advertencia sonora/bocina de tren en aproximación.
Obstáculos	hurdle_pass.wav	Zumbido de aire cercano al superar una valla.
Coleccionables	coin_pickup_1.wav .. 10.wav	Recolección de monedas con incremento tonal en escala.
Potenciadores	powerup_magnet_loop.wav	Zumbido electromagnético constante mientras el Imán está activo.
Potenciadores	powerup_jetpack_ignite.wav	Ignición y despegue de la mochila cohete.
Potenciadores	powerup_jetpack_loop.wav	Propulsión continua del Jetpack en fase aérea.
Potenciadores	hoverboard_activate.wav	Despliegue del Hoverboard bajo los pies del personaje.
Potenciadores	hoverboard_break.wav	Disipación de energía y destrucción del Hoverboard tras un impacto.
Persecución	inspector_step_heavy.wav	Pasos pesados y acelerados del inspector a corta distancia.
Persecución	dog_bark_close.wav	Ladridos y jadeo continuos del perro en estado de advertencia.
Colisiones	impact_fatal_train.wav	Impacto directo y violento contra estructura metálica rígida.

Colisiones	impact_stumble_wood.wav	Tropezón rozante contra madera o barrera ligera.
Interfaz (UI)	ui_menu_browse.wav	Modificación de foco o desplazamiento en elementos de menú.
Interfaz (UI)	ui_game_over.wav	Despliegue del resumen de puntuación al ser atrapado.

Guía Paso a Paso para la Extracción de Recursos del APK Original (2012)

El paquete instalador de *Subway Surfers* para Android (compilaciones del año 2012 desarrolladas sobre el motor Unity) almacena sus efectos de sonido dentro de contenedores binarios .assets o .resource¹. El procedimiento de extracción se realiza conforme a los siguientes pasos:

1. **Obtención del Paquete APK:** Conseguir el paquete oficial ejecutable .apk correspondiente a las versiones iniciales de 2012 (como v1.0.0 a v1.4.0) a través de servicios de preservación de software.
2. **Descompresión del Contenedor:** Alterar la extensión del archivo de .apk a .zip y descomprimir su estructura de carpetas mediante una herramienta de extracción de archivos.
3. **Localización de los Bundles de Unity:** Navegar a la carpeta contenedora de recursos del motor Unity situada en la ruta: \assets\bin\Data\ En dicha ubicación se identifican los archivos binarios sharedassets0.assets, sharedassets1.assets o resources.assets.
4. **Carga en Herramientas de Inspección:**
 - Ejecutar la aplicación **AssetStudioGUI** (o **AssetRipper**) en un entorno Windows⁶.
 - Acceder al menú File -> Load Folder y seleccionar la carpeta \assets\bin\Data\ que fue descomprimida previamente⁶.
5. **Filtrado y Exportación de Clips de Audio:**
 - Una vez procesada la tabla de contenidos, acceder a la pestaña Asset List.
 - En el filtro de tipos (Filter Type), marcar exclusivamente la categoría AudioClip.
 - Seleccionar la totalidad de los archivos de audio desplegados, hacer clic derecho y marcar la opción Export Selected Assets.
 - Indicar una carpeta de salida. La herramienta convertirá y guardará los flujos sonoros en formato .wav descomprimido o .ogg⁶.
6. **Procesamiento de Archivos FSB (FMOD Sound Bank):** Si la extracción genera contenedores intermedios .fsb, se debe emplear la utilidad de línea de comandos python-fsb5 para descomprimir las muestras individuales .wav:

DOS

vimon-m /sb5-e-o /sbx-extraidos/contenedor-audio-isl

6

Fuentes Secundarias y Bancos de Sonido Alternativos

En el caso de que ciertos clips extraídos presenten música de fondo integrada o degradación por compresión, se debe recurrir a repositorios de audio profesional bajo licencias abiertas:

- **Freesound.org**: Plataforma ideal para la búsqueda de sonidos Foley aislados, como pisadas sobre balasto seco, arrastre de calzado y chasquidos de madera.
- **BBC Sound Effects Archive**: Colección que ofrece registros de alta fidelidad para pisadas pesadas con calzado rígido y vocalizaciones de caninos en movimiento.
- **Sonniss GDC Audio Archives**: Bibliotecas de grado comercial distribuidas anualmente que contienen grabaciones de pasadas de trenes diésel a alta velocidad, motores ferroviarios e impactos metálicos secos.

3. Diseño de Audio Binaural con HRTF (Espacialización 3D Realista)

Mapeo de Azimut para la Ubicación de Carriles

La simulación de audio tridimensional mediante Funciones de Transferencia Relacionadas con la Cabeza (HRTF) procesa las pistas de Tiempo Interaural (ITD) y Nivel Interaural (ILD) para posicionar fuentes sonoras en el plano horizontal respecto al oyente. El cálculo del ángulo de

azimut (θ) para un objeto situado en el espacio de juego se obtiene mediante la relación vectorial entre el emisor y el receptor:

$$\theta = \arctan 2(X_{\text{objeto}} - X_{\text{jugador}}, Z_{\text{objeto}} - Z_{\text{jugador}}) \quad (1)$$

Elemento / Fuente	Coordenada X	Azimut (θ) a 10m de Distancia	Azimut (θ) a 3m de Distancia	Percepción Psicoacústica Binaural
Carril Izquierdo	-3.0 m	-16.7°	-45.0°	Predominio de llegada previa e intensidad en oído izquierdo.

Carril Central	0.0 m	0.0°	0.0°	Amplitud y fase idénticas en ambos oídos (frente exacto).
Carril Derecho	+3.0 m	+16.7°	+45.0°	Predominio de llegada previa e intensidad en oído derecho.
Inspector / Perro	0.0 m	180.0°	180.0°	Filtrado de sombra posterior de la pinna (atenuación de agudos).

El posicionamiento binaural del espacio tridimensional sitúa el plano frontal en los 0° de azimut, alineado con el carril central. Los carriles izquierdo y derecho generan ángulos de azimut negativos y positivos respectivamente, cuya separación relativa se amplifica a medida que los objetos se aproximan al avatar. El plano de persecución se establece en el azimut plano de 180° (atrás exacto), aplicando el perfil HRTF correspondiente a la atenuación que produce el pabellón auricular en los sonidos originados a la espalda del oyente.

Mapeo de Elevación y Coordenada Vertical (Z / Y)

La detección de elevación en el sistema auditivo humano depende de las muescas e impresiones espectrales que el pabellón auricular introduce en el rango de frecuencias comprendido entre 4 kHz y 10 kHz . En el motor de juego, se implementa una modulación combinada de elevación HRTF (ϕ) y filtros DSP activos⁸:

- **Salto sobre Valla Baja o con Super Sneakers:**
 - **Coordenada de Elevación:** Modulación dinámica del ángulo de elevación de $\phi = 0^\circ$ a $\phi = +45^\circ$ conforme el personaje alcanza el ápex del salto.
 - **Procesamiento DSP:** Aplicación de un filtro *High-Shelf* con una ganancia de $+3 \text{ dB}$

concentrada en los 6 kHz durante la elevación, acompañado por un ligero incremento tonal ascendente (*Pitch Shift* de 1.0 a 1.15).

- **Rodar bajo Valla Alta:**

- **Coordenada de Elevación:** Desplazamiento del ángulo de elevación de $\phi = 0^\circ$ a $\phi = -35^\circ$.

- **Procesamiento DSP:** Filtro paso bajo (*Low-Pass Filter*) de segundo orden con una frecuencia de corte descendente hasta $f_c = 3500 \text{ Hz}$ en el punto más bajo del rodamiento, simulando la cercanía al suelo y la oclusión del cuerpo.

- **Navegación sobre Techo de Trenes:**

- **Coordenada de Elevación:** Se fija un ángulo continuo de elevación de $\phi = +22^\circ$ mientras se permanezca sobre la cubierta del vagoneta.
- **Cambio de Textura Foley:** Se conmuta de forma inmediata el banco de pisadas de balasto a pisadas metálicas estruendosas (*step_metal_1.wav*). Todos los eventos situados al nivel de las vías adoptan un ángulo de elevación descendente ($\phi = -25^\circ$).

Filtrado por Oclusión, Atenuación y Efecto Doppler

Para proyectar una física acústica coherente, el motor procesa tres algoritmos de DSP en tiempo real:

Atenuación por Distancia

Se implementa un modelo de atenuación logarítmica inversa con una distancia mínima de referencia ($d_{\min} = 2.0 \text{ m}$) y una distancia máxima de audibilidad ($d_{\max} = 50.0 \text{ m}$):

$$A(d) = \begin{cases} 1.0 & \text{si } d \leq d_{\min} \\ \frac{d_{\min}}{d_{\min} + k \cdot (d - d_{\min})} & \text{si } d_{\min} < d \leq d_{\max} \\ 0.0 & \text{si } d > d_{\max} \end{cases} \quad (1)$$

En esta función, k es el factor de caída configurado en 1.2 para mantener un gradiente de volumen claro pero no abrupto.

Oclusión Acústica por Obstáculos

Cuando una vagoneta estática se interpone en la línea recta de visión entre el emisor (por ejemplo, una moneda o valla) y la posición del jugador, se aplica un filtro paso bajo Butterworth

de segundo orden. La frecuencia de corte se degrada proporcionalmente al nivel de oclusión:

$$f_c = 20000 \cdot (1.0 - 0.75 \cdot \text{FactorOclusión}) \quad (1)$$

Esta pérdida de altas frecuencias proporciona la pista auditiva de que el objeto objetivo carece de una línea de avance directa y se halla oculto tras una estructura.

Efecto Doppler para Trenes en Aproximación

La velocidad relativa de acercamiento de los trenes en movimiento demanda una modulación de frecuencia para reflejar la compresión de la onda sonora⁸. La frecuencia percibida f' se calcula dinámicamente mediante la ecuación clásica de Doppler:

$$f' = f \cdot \left(\frac{c + v_{\text{oyente}}}{c - v_{\text{fuente}}} \right) \quad (1)$$

Donde $c = 343 \text{ m/s}$ representa la velocidad del sonido en el aire. Para una velocidad combinada de 42.0 m/s , la relación de tono sube a 1.14 durante la aproximación (tono agudo) y cae bruscamente a 0.89 al sobrepasar al oyente, generando la alerta psicoacústica de un objeto pesado desplazándose a gran velocidad hacia la posición del jugador⁸.

4. Integración de Accesibilidad y Sugerencias para Windows con NVDA (.NET / C#)

Arquitectura de Conexión entre C# .NET y NVDA

El desarrollo del juego en C# .NET requiere conectar el bucle principal de audio con el lector de pantalla activo en el sistema operativo⁸. Aunque se puede realizar una llamada directa P/Invoke a la librería nvdaControllerClient.dll nativa de NVDA, la arquitectura más sólida para software accesible en Windows utiliza capas de abstracción como **Tolk**⁹ o **SpeechCore**¹¹.

El flujo de información en la arquitectura de software se articula en capas independientes para no comprometer el rendimiento:

1. **Capa del Motor de Juego (C# .NET):** Procesa la lógica de posiciones, velocidades, colisiones e inventario a 60 fotogramas por segundo.
2. **Capa de Renderizado Audio 3D:** Recibe coordenadas de objetos del motor y procesa los clips mediante un motor espacializador (como OpenAL Soft o FMOD Studio) aplicando las tablas HRTF.
3. **Capa Abstracción de Accesibilidad (Tolk / SpeechCore):** Captura eventos de la

interfaz (menús, estado del juego, logros) y los envía a la DLL del lector activo (nvdaControllerClient.dll, SAAPI64.dll para System Access, o drivers de JAWS)⁸.

4. **Lector de Pantalla de Windows:** Ejecuta la síntesis de voz seleccionada por el usuario para anunciar la información contextual sin interferir con las fuentes de audio del juego.

Estrategia de Verbalización de la Interfaz y Coexistencia de Audio

El flujo de voz accesibilidad no debe eclipsar la mezcla auditiva binaural ni saturar el canal de audio del jugador. Se establecen tres directrices de implementación:

1. **Llamadas Asíncronas No Bloqueantes:** Todas las invocaciones a las funciones de habla de la API de NVDA o Tolk (Tolk_Speak) deben realizarse de forma asíncrona⁹. Esto evita que el hilo de ejecución principal de la física o el motor de audio experimente pausas por cuellos de botella en la síntesis de voz⁹.
2. **Atenuación Automática de Audio (Audio Ducking):** Al emitir mensajes hablados extensos (como la lectura del menú de opciones o el desglose de estadísticas al perder), la mezcla master del audio 3D del juego debe atenuarse automáticamente entre -6 dB y -10 dB . El tiempo de ataque debe ser de 10 ms y el tiempo de liberación de 200 ms .
3. **Manejo Diferenciado de Interrupciones de Voz:**
 - *Navegación de Menús:* Se requiere la cancelación inmediata del mensaje anterior (interrupt = true) para permitir un desplazamiento rápido mediante flechas¹¹.
 - *Notificaciones en Vivo:* Se configura una emisión sin interrupción (interrupt = false), dejando que los efectos binaurales mantengan la máxima prioridad auditiva y reservando la voz únicamente para consultas explícitas de estado impulsadas por el jugador¹¹.

Alternativa de Auto-Voz (Self-Voicing Fallback)

En entornos donde el jugador no disponga de un lector de pantalla iniciado, la aplicación debe detectar la ausencia del servicio invocando Tolk_HasSpeech()⁹. Al retornar un valor falso, el motor en C# conmuta automáticamente hacia un subsistema de auto-voz basado en la API nativa SAPI5 de Windows mediante el espacio de nombres System.Speech.Synthesis:

```
C#
using System;
using System.Speech.Synthesis;

public class AccessibilityEngine
{
    private SpeechSynthesizer sapiVoice;
```

```

private bool IsScreenReaderActive

public void InitializeAccessibility()
{
    // Carga de la abstracción Tolk para detectar lectores activos
    Tolk.Load();
    IsScreenReaderActive = Tolk.HasSpeech();

    if (!IsScreenReaderActive)
    {
        // Inicialización de SAPI5 como mecanismo de auto-voz
        sapiVoice = new SpeechSynthesizer();
        sapiVoice.SetOutputToDefaultAudioDevice();
        sapiVoice.Rate = 2 // Incrementar velocidad de habla para fluidez
    }
}

public void OutputText(string text, bool interrupt = false)
{
    if (IsScreenReaderActive)
    {
        Tolk.Output(text, interrupt);
    }
    else if (sapiVoice != null)
    {
        if (interrupt)
        {
            sapiVoice.SpeakAsyncCancelAll();
        }
        sapiVoice.SpeakAsync(text);
    }
}

public void Shutdown()
{
    Tolk.Unload();
    sapiVoice?.Dispose();
}

```

Maapeo de Controles Optimizado para Teclado en Windows

La configuración de controles se ha diseñado respetando las convenciones ergonómicas de

acceso por teclado a dos manos:

Tecla / Combinación	Acción Mecánica en el Juego	Respuesta Auditiva y Sintetizador
Flecha Izquierda / A	Desplazamiento al carril izquierdo.	Cue rápido de cambio de carril panreado a la izquierda.
Flecha Derecha / D	Desplazamiento al carril derecho.	Cue rápido de cambio de carril panreado a la derecha.
Flecha Arriba / W	Saltar (valla baja / subida a tren).	Tono dinámico ascendente espacializado en HRTF.
Flecha Abajo / S	Rodar / Agacharse (valla alta).	Tono dinámico descendente procesado con filtro paso bajo.
Barra Espaciadora	Activar Hoverboard de protección.	Sonido de despliegue energético metálico.
Tecla L	Consulta de carril actual.	Anuncio por síntesis: " <i>Carril Izquierdo</i> ", " <i>Centro</i> " o " <i>Derecho</i> ".
Tecla C	Consulta de monedas recolectadas.	Anuncio por síntesis: " <i>Monedas: [Cantidad]</i> ".
Tecla S	Consulta de metros/puntuación.	Anuncio por síntesis: " <i>Puntuación: [Metros]</i> ".
Tecla P	Consulta de potenciadores activos.	Anuncio por síntesis: " <i>Imán activo, 5 segundos restantes</i> ".
Shift + S	Consulta de récord personal.	Anuncio por síntesis: " <i>Máxima puntuación:</i>

		[Puntos]".
Escape	Pausar / Abrir Menú Principal.	Pausa del renderizado 3D, atenuación y lectura de interfaz ¹¹ .

5. Conclusiones e Impacto del Diseño

El diseño de una versión en formato audiogame binaural para *Subway Surfers* (2012) estructurada en C# .NET demuestra la viabilidad técnica de adaptar títulos de acción continua en tiempo real para usuarios ciegos o con discapacidad visual. La reconstrucción precisa de las variables dinámicas originales —como el modelo de tres carriles discretos, la curva logarítmica de aceleración¹ y el dimensionamiento físico de los obstáculos— proporciona la base sobre la cual se asienta el sistema de representación sonora.

El uso de un procesamiento binaural sustentado en HRTF, enriquecido mediante atenuación por distancia, filtros de oclusión Butterworth y modelado de desplazamiento Doppler⁸, permite transformar la información espacial visual en pistas psicoacústicas intuitivas. El jugador no requiere simplificaciones en la jugabilidad; por el contrario, la respuesta espacial del entorno de audio le otorga la capacidad de anticipar y reaccionar ante obstáculos dentro de ventanas

temporales de entre **400 ms** y **667 ms** con absoluta precisión.

Por último, la arquitectura de accesibilidad propuesta, que integra capas universales de abstracción para lectores de pantalla como Tolk⁹ e interfaces nativas de NVDA, junto con mecanismos de auto-voz (SAPI5) y un esquema de control por teclado ergonómico, establece un marco de trabajo sólido, extensible y de alto rendimiento para el desarrollo de audiogames accesibles en el ecosistema Windows.

Obras citadas

1. Make Your Own Runner Game - Unity, <https://unity.com/features/build-a-runner-game>
2. The top most viral and addictive casual games - ArionisGames, <https://arionisgames.com/blog/top-casual-games/>
3. Subway Surfers (Video Game) - TV Tropes, <https://tvtropes.org/pmwiki/pmwiki.php/VideoGame/SubwaySurfers>
4. endless runner - RSSing.com, https://endless64.rssing.com/chan-23159273/all_p1.html
5. Steam Workshop::гшмммод чувр, <https://steamcommunity.com/sharedfiles/filedetails/?id=2982825152>
6. extract audio files from unity assets within an android obb file - GitHub Gist, <https://gist.github.com/kibotu/fb5a9ce6c8820727280d31c852669bca>
7. How to study Unity Games on Steam with ILSpy and AssetStudio - YouTube, <https://www.youtube.com/watch?v=S6vS8DmlyYU>
8. AccessMods - GitHub, <https://github.com/AccessMods>

9. dkager/tolk: Screen reader abstraction library. - GitHub,
<https://github.com/dkager/tolk>
10. khanshoaib3/CrossSpeak: A cross platform screen reader interfacing library. -
GitHub, <https://github.com/khanshoaib3/CrossSpeak>
11. GitHub - still-standing88/SpeechCore: C++ cross-platform speech library,
<https://github.com/still-standing88/SpeechCore>
12. oasis1701/footballmanager-accessibility - GitHub,
<https://github.com/oasis1701/footballmanager-accessibility>